Mobile Application Development

Fall 2025

Lecture : 22/10/2025

KOTLIN – Lecture 2

Functions

• A Kotlin function is declared using the following syntax:

```
fun <function name> (<para name>: <para type>, <para name>: <para type>,... ): <return type> {
//
} Function code

fun sayHello() {
println("Hello")
}

fun buildMessageFor(name: String, count: Int): String {
return("$name, you are customer number $count")
}
```

Calling a Kotlin function

<function name> (<arg1>, <arg2>, ...)

• Each argument passed through to a function must match the parameters the function is configured to accept.

```
fun multiply(x: Int, y: Int): Int {
return x * y
}
```

• Same function expressed as a single line expression:

```
fun multiply(x: Int, y: Int): Int = x * y
```

Local Functions

- A local function is a function that is embedded within another function.
- A local function has access to all of the variables contained within the enclosing function.

```
fun main(args: Array<String>) {
val name = “KS"
val count = 5
fun displayString() {
        for (index in 0..count) {
        println(name)
        }
}
displayString()
}
```

Handling Return Values

- To call a function named buildMessage that takes two parameters and returns a result, on the other hand, we might write the following code:

val message = buildMessageFor("John", 10)

- To improve code readability, the parameter names may also be specified when making the function call:

val message = buildMessageFor(name = "John", count = 10)

- Created a new variable called message and then used the assignment operator (=) to store the result returned by the function.

Declaring default function parameters

- Kotlin provides the ability to designate a default parameter value to be used if the value is not provided as an argument when the function is called.
- This simply involves assigning the default value to the parameter when the function is declared.

```
fun main(args: Array<String>) {
        fun buildMessageFor(name: String = "Customer" , count: Int = 0 ):
String        {
        return("$name, you are customer number $count")
        }
        println(buildMessageFor())
  }
```

- *buildMessageFor* function will be modified so that the string “Customer” is used as a default if a customer name is not passed through as an argument.
- Similarly, the *count* parameter is declared with a default value of 0

- When parameter names are used when making the function call, any parameters for which defaults have been specified may be omitted.

```
val message = buildMessageFor(count = 10)
```

- omits the customer name argument but still compiles because the parameter name has been specified for the second argument

- If parameter names are not used within the function call, only the trailing arguments may be omitted:

```
val message = buildMessageFor("John") // Valid
val message = buildMessageFor(10) // Invalid
```

Variable Number of Arguments for a function

- It is not always possible to know in advance the number of parameters a function will need to accept when it is called within the application code.
- Kotlin handles this possibility through the use of the *vararg* keyword to indicate that the function accepts an arbitrary number of parameters of a specified data type.
- Within the body of the function, the parameters are made available in the form of an array object.

```
fun displayStrings(vararg strings: String) {
        for (string in strings) {
                println(string)
        }
}
displayStrings("one", "two", "three", "four")
```

- Kotlin does not permit multiple vararg parameters within a function and any single parameters supported by the function must be declared before the vararg declaration:

```
fun displayStrings(name: String, vararg strings: String) {
        for (string in strings) {
                println(string)
        }
}
```

Lambda expressions

- lambdas are self-contained blocks of code.
- The following code, for example, declares a lambda, assigns it to a variable named sayHello, and then calls the function via the lambda reference:

```
val sayHello = { println("Hello") }
sayHello()
```

- Lambda expressions may also be configured to accept parameters and return results.

```
{<para name>: <para type>, <para name> <para type>, ... ->
// Lambda expression here
}
```

- The following lambda expression, accepts two integer parameters and returns an integer result:

```
val multiply = { val1: Int, val2: Int -> val1 * val2 }
val result = multiply(10, 20)
```

Lambda – Function Assignments

- Previous lambda examples have assigned the lambda code block to a variable.
- This is also possible when working with functions.
- Following syntax will execute the function and assign the result of that execution to a variable, instead of assigning the function itself to the variable:

```
val myvar = myfunction()
```

- To assign a function reference to a variable, simply remove the parentheses and prefix the function name with double colons (::) ….
- The function may then be called simply by referencing the variable name:

```
val myvar = ::myfunction
myvar()
```

Lambda block execution

- A lambda block may be executed directly by placing parentheses at the end of the expression including any arguments.
- The following lambda directly executes the multiplication lambda expression multiplying 10 by 20

```
val result = { val1: Int, val2: Int -> val1 * val2 }(10, 20)
```

- The last expression within a lambda serves as the expression's return value …. the value of 200 being assigned to the result variable in the above multiplication examples.

- Unlike functions, lambdas do not support the *return* statement.
- In the absence of an expression that returns a result (such as an arithmetic or comparison expression), simply declaring the value as the last item in the lambda will cause that value to be returned.
- The following lambda returns the Boolean true value after printing a message:

```
val result = { println("Hello"); true }()
```

- Similarly, the following lambda simply returns a string literal:

```
val nextmessage = { println("Hello"); "Goodbye" }()
```

- ; optional statement separator … usually in a block next line indicates new statement

Higher-order functions

- A function that is capable of receiving a function or lambda as an argument, or returning one as a result is referred to as a *higher-order function*.

- The type of a function is dictated by a combination of the parameters it accepts and the type of result it returns.
- A function that accepts an Int and a Double as parameters and returns a String result for example is considered to have the following function type:

(Int, Double) -> String

- To accept a function as a parameter, the receiving function simply declares the type of function it can accept.
- E.g. we declare two unit conversion functions:

```
fun inchesToFeet (inches: Double): Double {
return inches * 0.0833333
}

fun inchesToYards (inches: Double): Double {
return inches * 0.0277778
}
```

- To demonstrate functions as parameters, need new function that will take as a parameter a function type that matches both the inchesToFeet and inchesToYards functions together with a value to be converted.
- Since the type of these functions is equivalent to (Double) -> Double, our general-purpose function can be written as follows:

```
fun outputConversion(converterFunc: (Double) -> Double, value: Double) {
        val result = converterFunc(value)
        println("Result of conversion is $result") }
```

- When the outputConversion function is called, it will need to be passed a function matching the declared type.
- That function will be called to perform the conversion and the result displayed in the console panel.
- This means that the same function can be called to convert inches to both feet and yards, simply by “plugging in” the appropriate converter function as a parameter, keeping in mind that it is the function reference that is being passed as an argument:

```
outputConversion(::inchesToFeet, 22.45)
outputConversion(::inchesToYards, 22.45)
```

- Functions can also be returned as a data type simply by declaring the type of the function as the return type.
- The following function is configured to return either our inchesToFeet or inchesToYards function type (in other words, a function that accepts and returns a Double value) based on the value of a Boolean parameter:

```
fun decideFunction(feet: Boolean): (Double) -> Double {
        if (feet) {
                return ::inchesToFeet
        } else {
                return ::inchesToYards
        }
}
```

- When called, the function will return a function reference which can then be used to perform the conversion:

```
val converter = decideFunction(true)
val result = converter(22.4)
println(result)
```

CLASSES

- The basic syntax for a new class is as follows:

```
class NewClassName: ParentClass {
// Properties
// Methods
}
```

Adding properties to a class

```
class BankAccount {
var accountBalance: Double = 0.0
var accountNumber: Int = 0
}
```

Adding Methods

```
class BankAccount {
        var accountBalance: Double = 0.0
        var accountNumber: Int = 0
        fun displayBalance()
        {
        println("Number $accountNumber")
        println("Current balance is $accountBalance")
        }
}
```

- Methods are declared within the opening and closing braces of the class to which they belong and are declared using the standard Kotlin function declaration syntax.

Declaring and initializing a class instance

- The first step in this process is to declare a variable to store a reference to the instance when it is created. We do this as follows:

```
val account1: BankAccount = BankAccount()
```

- When executed, an instance of our BankAccount class will have been created and will be accessible via the account1 variable.
- Of course, the Kotlin compiler will be able to use inference here, making the type declaration optional:

```
val account1 = BankAccount()
```

Primary and secondary constructors

- Class will often need to perform some initialization tasks at the point of creation.
- These tasks can be implemented using constructors within the class.
- In the case of the BankAccount class, it would be useful to be able to initialize the account number and balance properties with values when a new class instance is created.
- To achieve this, a *secondary constructor* can be declared within the class header

Constructor

```
class BankAccount {
var accountBalance: Double = 0.0
var accountNumber: Int = 0
constructor(number: Int, balance: Double) {
accountNumber = number
accountBalance = balance
}.
.
}

val account1: BankAccount = BankAccount(456456234, 342.98)
```

Additional secondary constructors

- Multiple secondary constructors allows instances of the class to be initiated with different value sets.
- E.g. when initializing an instance with the customer's last name in addition to the corresponding account number and balance:

```
class BankAccount {
var accountBalance: Double = 0.0
var accountNumber: Int = 0
var lastName: String = ""
constructor(number: Int,
balance: Double) {
accountNumber = number
accountBalance = balance
}
constructor(number: Int,
balance: Double,
name: String ) {
accountNumber = number
accountBalance = balance
lastName = name
}.
.
}
```

- Instances of the BankAccount may now also be created as follows:

```
val account1: BankAccount = BankAccount(456456234, 342.98
```

- It is also possible to use a *primary constructor* to perform basic initialization tasks.
- The primary constructor for a class is declared within the class header as follows:

```
class BankAccount (val accountNumber: Int, var accountBalance: Double)
{.
.
fun displayBalance()
{
println("Number $accountNumber")
println("Current balance is $accountBalance")
}
}
```

Both properties have been declared in the primary constructor, it is no longer necessary to also declare the variables within the body of the class.
Since the account number will now not change after an instance of the class has been created, this property is declared as being immutable using the *val* keyword.

- A class may only contain one primary constructor, Kotlin allows multiple secondary constructors to be declared in addition to the primary constructor

```
class BankAccount (val accountNumber: Int, var accountBalance: Double) {
var lastName: String = ""
constructor(accountNumber: Int, accountBalance: Double, name:
String ) : this(accountNumber, accountBalance) {
lastName = name
}.
.
}
```

- Although the accountNumber and accountBalance properties are accepted as parameters to the secondary constructor, the variable declarations are still handled by the primary constructor and do not need to be declared.
- To associate the references to these properties in the secondary constructor with the primary constructor, however, they must be linked back to the primary constructor using the *this* keyword:
- ... this(accountNumber, accountBalance)...

Initializer blocks

- Primary constructor cannot contain any code, these methods are a particularly useful location for adding code to perform initialization tasks when an instance of the class is created.
- Initializer blocks are declared using the *init* keyword with the initialization code enclosed in braces:

```
class BankAccount (val accountNumber: Int, var accountBalance: Double) {
init {
// Initialization code goes here
}.
.
}
```

Calling methods and accessing properties

- Dot notation involves accessing a property, or calling a method by specifying a class instance followed by a dot followed in turn by the name of the property or method:

classInstance.propertyname

classInstance.methodname()

- For example, to get the current value of accountBalance instance variable:

val balance1 = account1.accountBalance

- Dot notation can also be used to set values of instance properties:

account1.accountBalance = 6789.98

- The same technique is used to call methods on a class instance

account1.displayBalance()

Custom accessors

- When accessing the accountBalance property in the previous section, the code is making use of property accessors that are provided automatically by Kotlin.
- In addition to these default accessors, it is also possible to implement *custom accessors* that allow calculations or other logic to be performed before the property is returned or set.
- Custom accessors are implemented by creating getter and optional corresponding setter methods containing the code to perform any tasks before returning the property.

```
class BankAccount (val accountNumber: Int, var accountBalance: Double) {
val fees: Double = 25.00
val balanceLessFees: Double
get() {
return accountBalance - fees
}f
un displayBalance()
{
println("Number $accountNumber")
println("Current balance is $accountBalance")
}
}
```

- code adds a getter that returns a computed property based on the current balance minus a fee amount.

- An optional setter could also be declared in much the same way to set the balance value less fees.
- The new setter takes as a parameter a Double value from which it deducts the fee value before assigning the result to the current balance property.

```
val fees: Double = 25.00
var balanceLessFees: Double
get() {
return accountBalance - fees
}
set(value) {
accountBalance = value - fees
}.
.
}
```

```
val balance1 = account1.balanceLessFees
account1.balanceLessFees = 12123.12
```

Nested and Inner Classes

- Kotlin allows one class to be nested within another class.
- For example, ClassB is nested inside ClassA:

```
class ClassA {
class ClassB {
}
}
```

```
class ClassA {
var myProperty: Int = 10
inner class ClassB {
val result = 20 + myProperty
}
}
```

- ClassB does not have access to any of the properties within the outer class.
- If access is required, the nested class must be declared using the *inner* directive.

Companion objects

- A Kotlin class can also contain a companion object.
- A companion object contains methods and variables that are common to all instances of the class.
- In addition to being accessible via class instances, these properties are also accessible at the class level (in other words without the need to create an instance of the class).

```
class ClassName: ParentClass {
// Properties
// Methods
companion object {
// properties
// methods
}
}
```

```
class MyClass {
fun showCount() {
println("counter = " + counter)
}
companion object {
var counter = 1
fun counterUp() {
counter += 1
}
}
}
fun main(args: Array<String>) {
println(MyClass.counter)
}
```

- class contains a companion object consisting of a counter variable and a method to increment that variable.
- The class also contains a method to display the current counter value.
- The *main()* method simply displays the current value of the counter variable, but does so by calling the method on the class itself instead of a class instance:

println(**MyClass** .counter)

Inheritance

Sub-Classes

- By deriving classes, we create what is often referred to as a class hierarchy.
- The class at the top of the hierarchy is known as the base class or root class and the derived classes as subclasses or child classes.
- Any number of subclasses may be derived from a class.
- The class from which a subclass is derived is called the parent class or superclass.
- Classes need not only be derived from a root class.

Subclassing syntax

- As a safety measure designed to make Kotlin code less prone to error, before a subclass can be derived from a parent class, the parent class must be declared as open.
- This is achieved by placing the *open* keyword within the class header:

```
open class MyParentClass {
    var myProperty: Int = 0
}
```

- With a simple class of this type, the subclass can be created as follows:

```
class MySubClass : MyParentClass() {
}
```

Primary or Secondary constructors

- For classes containing primary or secondary constructors, the rules for creating a subclass are slightly more complicated!
- Consider the following parent class which contains a primary constructor:

```
open class MyParentClass(var myProperty: Int) {
}
```

- To create a subclass of this class, the subclass declaration references any base class parameters while also initializing the parent class using the following syntax:

class MySubClass(**myProperty: Int**) : MyParentClass(**myProperty**) {

}

- If the parent class contains one or more secondary constructors, the constructors must also be implemented within the subclass declaration and include a call to the secondary constructors of the parent class, passing through as arguments the values passed to the subclass secondary constructor.
- When working with subclasses, the parent class can be referenced using the *super* keyword.
- A parent class with a secondary constructor might read as follows:

```
open class MyParentClass {
    var myProperty: Int = 0
    constructor(number: Int) {
        myProperty = number
    }
}
```

- The code for the corresponding subclass would need to be implemented as follows:

```
class MySubClass : MyParentClass {
constructor(number: Int) : super(number)
}
```

- If additional tasks need to be performed within the constructor of the subclass, this can be placed within curly braces after the constructor declaration:

```
class MySubClass : MyParentClass {
    constructor(number: Int) : super(number) {
        // Subclass constructor code here
    }
}
```

Inheritance Example

- The BankAccount class (Prev. Slides) contain both properties and methods.
- A simplified declaration for this class is reproduced below and will be us

```
class BankAccount {
var accountNumber = 0
var accountBalance = 0.0
constructor(number: Int, balance: Double) {
accountNumber = number
accountBalance = balance
}
open fun displayBalance() ed for the basis of the subclassing example {
println("Number $accountNumber")
println("Current balance is $accountBalance")
}
}
```

Saving Account - SubClass

- Suppose that in addition to the BankAccount class you also needed a class to be used for savings accounts.
- A savings account will need to hold an account number and a current balance and methods will still be needed to access that data.
- Creating a new class that is a subclass of the BankAccount class!
- The new class will then inherit all the features of the BankAccount class but can then be extended to add the additional functionality required by a savings account.

- Before a subclass of the BankAccount class can be created, the declaration needs to be modified to declare the class as open:

open class BankAccount {

- To create a subclass of BankAccount that we will call SavingsAccount, we simply declare the new class, this time specifying BankAccount as the parent class and add code to call the constructor on the parent class:

```
class SavingsAccount : BankAccount {
constructor(accountNumber: Int, accountBalance: Double) :
super(accountNumber, accountBalance)
}
```

- We have yet to add any properties or methods, the class has inherited all the methods and properties of the parent BankAccount class.

Extend subclass

- Extend the subclass so that it has the features we need to make it useful for storing savings account information.
- To do this, we simply add the properties and methods that provide the new functionality, just as we would for any other class we might wish to create:

```
class SavingsAccount : BankAccount {
    var interestRate: Double = 0.0
    constructor(accountNumber: Int, accountBalance: Double) :
                                        super(accountNumber, accountBalance)
   fun calculateInterest(): Double
   {
       return interestRate * accountBalance
   }
}
```

Overriding

Three rules that must be obeyed when overriding a method.

- First, the overriding method in the subclass must accept the same number and type of parameters as the overridden method in the parent class.
- Second, the new method must have the same return type as the parent method.
- Third, the original method in the parent class must be declared as open before the compiler will allow it to be overridden.

Override keyword

```
class SavingsAccount :
BankAccount {
var interestRate: Double = 0.0
constructor(accountNumber: Int,
accountBalance: Double) :
super(accountNumber,
accountBalance)
fun calculateInterest(): Double
{
return interestRate *
accountBalance
}
```

```
override fun displayBalance()
{
println("Number $accountNumber")
println("Current balance is
$accountBalance")
println("Prevailing interest rate is
$interestRate")
}}
```

Before this code will compile, the displayBalance method in

the BankAccount class must be declared as open:

```
open fun displayBalance()
{
println("Number $accountNumber")
println("Current balance is $accountBalance")
}
```

Before this code will compile, the displayBalance method in the BankAccount class must be declared as open:

```
override fun displayBalance()
{
super.displayBalance()
println("Prevailing interest rate is $interestRate")
}
```

• BankAccount class:

```
val savings1 = SavingsAccount(12311, 600.00, 0.07)
println(savings1.calculateInterest())
savings1.displayBalance()
```

- Download Git
- Commit Changes to Git
- Profile